

Launch Control XL Erklärung



bitte erkläre mir das dokument.

Das Dokument „**Launch Control XL Programmer’s Reference Guide**“ ist ein technisches Handbuch für Entwickler:innen, die den **Novation Launch Control XL** über MIDI oder System Exclusive (Sysex) programmieren möchten – also z.B. in **Max/MSP**, **Pure Data**, **Max for Live**, etc.

Hier sind die wichtigsten Punkte zusammengefasst:

Allgemeines Verständnis

- Das Launch Control XL ist ein **USB-MIDI-Controller** mit:
 - 24 Drehreglern (Pots)
 - 8 Fadern
 - 24 Buttons (davon 16 mit RGB-LEDs: Rot & Grün, die gemischt Amber ergeben können)
- Es gibt **16 Templates** (Voreinstellungen):
 - 8 **User-Templates** (änderbar, Slots 0–7)
 - 8 **Factory-Templates** (nicht änderbar, Slots 8–15)

MIDI-Kommunikation

Der Controller unterstützt zwei Protokolle:

1. Launchpad MIDI-Protokoll

- Verwendet **Note On/Off** und **Control Change (CC)**-Nachrichten
- Funktioniert **nur**, wenn der aktuelle Template-Slot mit dem MIDI-Wert übereinstimmt

2. System Exclusive (Sysex)

- Kann **beliebige LEDs** in **beliebigen Templates** ansteuern
- Empfohlen für verlässliche Kontrolle

LED-Steuerung

Eine LED wird über ein Byte gesteuert:

- **Helligkeit Rot (2 Bits):** 0 = aus, 1 = schwach, 2 = mittel, 3 = hell
- **Helligkeit Grün (2 Bits)**

- **Copy/Clear-Flags** für **Doppel-Buffering**

Beispielwerte:

Farbe	Hex	Dez
Aus	0C	12
Rot (voll)	0F	15
Grün (voll)	3C	60
Amber (voll)	3F	63

Double-Buffering & Blinken

Der Controller hat **zwei LED-Speicher (Buffer 0 & 1)**:

- Man schreibt in einen, zeigt den anderen → **flackerfreies Umschalten**
- Kann auch zum **automatischen Blinken** verwendet werden

Beispiel:

1. Setze Buffer 1 als sichtbar, Buffer 0 als beschreibbar → `Bnh, 00h, 31h`
2. Schreibe LED-Daten ohne Copy/Clear
3. Zeige Buffer 0 → `Bnh, 00h, 34h`
4. Wiederhole

Wichtige Sysex-Nachrichten

- **Setze LED:**

```
pgsql
`F0 00 20 29 02 11 78 Template Index Wert F7`
```

- **Schalte Button (Toggle):**

```
pgsql
`F0 00 20 29 02 11 7B Template Index Wert F7`
```

- **Template wechseln:**

```
nginx
`F0 00 20 29 02 11 77 Template F7`
```

Daten vom Gerät an den Computer

- Button gedrückt: MIDI Note On/Off oder CC (je nach Einstellung im Editor)
- Template-Wechsel wird auch per Sysex gemeldet

Wenn du mir sagst, wofür du das Dokument brauchst (z.B. Max for Live Device, Lichtsteuerung, Setup mit OSC etc.), kann ich dir ein konkretes Beispiel oder eine Anleitung daraus ableiten.



ich will midi daten an den controller senden

Perfekt – du willst also **vom Computer aus MIDI-Daten an den Novation Launch Control XL senden**, um z. B. **LEDs zu steuern**. Je nachdem, **welche Funktionalität** du willst, kannst du das entweder mit **klassischem MIDI** oder mit **Sysex-Nachrichten** tun.

◆ Option 1: Klassisches MIDI (Note On/CC) – einfach, aber limitiert

Funktioniert **nur**, wenn:

- Der aktuell gewählte **Template-Slot aktiv** ist
- Die CC- oder Notenummer genau **zum Button/Regler passt**, wie er im Editor eingestellt ist

Beispiel: LED per Note-On einschalten

Sende:

```
vbnet

`Status: 0x90 (MIDI-Kanal 1 → 0x90 = Note On, Channel 1)
Note: 32 (Button-Nummer)
Velocity: 63 (z.B. Amber Voll)
`
```

In Dezimal:

```
text

`144, 32, 63
```

✓ Das leuchtet den Button mit Note 32 in Amber auf voller Helligkeit, **wenn** der richtige Template aktiv ist.

◆ Option 2: System Exclusive (Sysex) – empfohlen, flexibel

Damit kannst du **jede LED jederzeit ansteuern**, unabhängig vom Template oder Note/CC-Einstellung.

Beispiel: LED über Sysex setzen

```
text
```

```
`F0 00 20 29 02 11 78 Template Index Wert F7`
```

Erklärung:

- `Template` = 0–7 (User-Templates), oder 8–15 (Factory)
- `Index` = Position des Knopfes (siehe Tabelle unten)
- `Wert` = Farbwert (z.B. 0Fh = Rot Voll = Dezimal 15)

Beispiel in Dezimal:

Setzt **Button 32** (untere Reihe, erster Channel-Button) auf **Amber voll** im **User-Template 0**:

```
text
```

```
`240, 0, 32, 41, 2, 17, 120, 0, 32, 63, 247`
```

1 2 3 4 Index-Tabelle für Buttons & Regler

Bereich	Indexbereich
Top-Row Regler	0–7
Mitte-Row Regler	8–15
Untere Regler	16–23
Obere Channel-Buttons	24–31
Untere Channel-Buttons	32–39
Device, Mute, Solo, Rec	40–43
Up, Down, Left, Right	44–47

✓ Empfohlenes Setup

Je nachdem, womit du arbeitest:

In Max/MSP:

```
max
```

```
`[ctlout]`  
[noteout]  
[midiout]  
`
```



ich verstehe sysex nicht. bitter erkläre mir wie es mit midi von max msp aus geht

Alles klar! Du brauchst also **keine Sysex**, sondern willst die **klassische MIDI-Steuerung aus Max/MSP** nutzen, um LEDs auf dem Launch Control XL zum Leuchten zu bringen. Super, das geht einfach und direkt – ich erklär dir das Schritt für Schritt.

Was du brauchst in Max/MSP

- ``noteout`` oder ``ctlout``
- Die richtigen **MIDI-Werte**
- Das richtige **Template auf dem Gerät aktiv**

♦ Beispiel 1: Button-LED mit ``noteout`` steuern

 **Ziel:** LED eines Buttons aufleuchten lassen

 **Max-Patch:**

```
max

`[144 32 63]    ← "Note On" auf MIDI-Kanal 1, Note 32, Velocity 63 (Amber voll)
|
[pack 0 0 0]
|
[noteout]
```

Oder noch einfacher:

```
max

`[144 32 63]
|
[midiout]
```

➡ Sende diesen Wert an den Launch Control XL – **die LED leuchtet, wenn du:**

- den richtigen Template (User 0–7) aktiv hast
- der Button wirklich Note 32 zugewiesen ist

♦ Beispiel 2: Button-LED ausschalten

```
max

`[144 32 0]
|
```

```
[midiout]
```

Velocity = 0 bedeutet: **aus**.

◆ Beispiel 3: LEDs mit `ctlout` steuern (wenn Buttons/Encoder per CC gesteuert werden)

```
max
`[176 21 63]
|
[midiout]
```

- 176 = **Control Change** auf Kanal 1
- 21 = **CC-Nummer**
- 63 = **Helligkeit/LED-Farbe**

🧠 Wichtig: CC geht nur, **wenn der Button in deinem Template als CC definiert ist**. (Das kannst du im Novation Editor überprüfen oder setzen.)

🔍 Wie findest du raus, welche Note oder CC ein Button sendet?

1. In Max:

```
max
`[notein]
|
[print]
```

oder

```
max
`[ctlin]
|
[print]
```

2. Drück einen Button – du siehst z.B.:

```
makefile
`notein: 32 127
```

Dann weißt du: **Note 32**, also kannst du dieselbe Note mit `noteout` zurückschicken, um die LED zu steuern.

🧠 Farben als MIDI-Velocity (aus dem PDF)

Farbe	Dezimal	Beschreibung
12	Aus	0C Hex
13	Rot schwach	
15	Rot voll	
29	Amber schwach	
63	Amber voll	
28	Grün schwach	
60	Grün voll	

Fazit

Du brauchst:

- ``noteout`` für Buttons, wenn sie mit Noten arbeiten
- ``ctlout`` für CCs
- **Velocity-Werte** steuern die LED-Farbe
- Stelle sicher, dass das **richtige Template** aktiv ist, und dass der Button den erwarteten MIDI-Wert hat

Willst du, dass ich dir ein kleines Max-Patch baue, das dir z. B. einen Button toggelt oder blinkt?



mit dem noteout kann ich auf jeden fall farben ändern wenn ich die erste und die 2. inlet des notwout objektes bediene. ich verstehe aber nicht was die werte genau machen und was der controller an werten brucht

Super, dass du schon mit ``noteout`` Farben auf dem Launch Control XL ändern kannst! Jetzt erklär ich dir **genau**, was die Werte am ``noteout`` machen – und **was der Controller braucht**, um LEDs gezielt zu steuern.



Aufbau von ``noteout``

max

``[noteout]`

Dieses Objekt hat 3 Inlets:

Inlet	Bedeutung
1	Pitch (Note-Nummer) = welcher Button

Inlet	Bedeutung
2	Velocity = Helligkeit & Farbe
3	Channel = MIDI-Kanal (0 = Kanal 1, 1 = Kanal 2, ...)

Der Controller braucht:

Ein **Note-On-Befehl** mit:

- einer **Note-Nummer** (welcher Button gemeint ist)
- einer **Velocity**, die eine **Farbe und Helligkeit** kodiert
- auf dem **richtigen MIDI-Kanal**

 **Hinweis:** Der MIDI-Kanal und die Note müssen **zum aktiven Template** und zum **Button-Mapping** passen. Du kannst das in der Novation Editor Software sehen oder ändern.

Velocity = Farbe

Der **Velocity-Wert** steuert **Farbe & Helligkeit**:

Dezimal	Farbe	Helligkeit
12	Aus	–
13	Rot	Schwach
15	Rot	Voll
28	Grün	Schwach
60	Grün	Voll
29	Amber	Schwach
63	Amber	Voll

(Amber = Mischung aus Rot + Grün)

Du kannst dir z. B. ein kleines Interface bauen, das dir Farben über Buttons schickt:

```
max

`[bang]
|
[pack 32 63 0] ← Button #32, Amber voll, MIDI Kanal 1
|
[noteout]
```

Testvorschlag: Welche Note ist welcher Button?

Du kannst mit ``notein`` testen, was du drücken musst:

```
max

`[notein]
```



```
[print]
```

- Drück einen Button auf dem Controller
- Die Konsole zeigt dir z. B.:

```
makefile
`notein: 32 127`
```

→ Du weißt jetzt: **Note 32** gehört zu diesem Button

Dann kannst du mit ``noteout`` zurückschicken:

```
max
`[32 63 0] → Amber voll auf Button 32 auf MIDI-Kanal 1`
```

Zusammengefasst: Was du brauchst

- ``noteout`` mit:
 - **Note** = Button-ID (z. B. 32)
 - **Velocity** = Farbwert (z. B. 63 = Amber voll)
 - **Kanal** = normalerweise 0 (für MIDI-Kanal 1)
- Aktives Template muss die Taste auf die Note hören

Wenn du willst, kann ich dir ein kleines Max-Patch machen, mit dem du Farben über Menüs auswählen und an Buttons senden kannst. Möchtest du das?



wenn kanal 1 bereits funktioniert, soll ich den kanal auch ändern ?

Wenn **MIDI-Kanal 1 (also Kanal 0 im ``noteout``-Objekt) funktioniert**, brauchst du **nichts zu ändern**. Dann ist alles richtig eingerichtet – bleib einfach dabei.

Warum gibt es mehrere MIDI-Kanäle?

Jeder Template-Slot (0–15) **kann seinen eigenen MIDI-Kanal verwenden**, aber:

- **User-Templates 0–7** sind meist auf **Kanal 1** voreingestellt
- Wenn du also aus Max ``noteout`` mit **Kanal 0 (also MIDI-Kanal 1)** schickst – und die LEDs reagieren – dann **passt das**

Empfehlung:

- Solange die Buttons mit `noteout` auf Kanal 0 (also MIDI-Kanal 1) blinken oder Farben ändern:
- ✓ alles gut – nicht ändern.

Nur wenn du:

- andere Templates gleichzeitig ansprechen willst
- Sysex verwenden willst, um mehrere Templates zu kontrollieren

... **dann** kann ein anderer Kanal relevant werden.

Wenn du magst, bau ich dir ein kleines Max-Beispiel mit `noteout`, Buttons und Farbwahl. Wäre das hilfreich für dich?



gut, also brauche ich nur die ersten beiden inlets von noteout

Ganz genau! 

Wenn du **nur einen MIDI-Kanal brauchst** (und z. B. **Kanal 1 funktioniert bei dir**), dann genügt es völlig, **nur die ersten beiden Inlets** von `noteout` zu verwenden:

✓ `noteout` in Max/MSP

max

`[noteout]` ← entspricht: MIDI-Kanal 1 (Standard, wenn drittes Inlet leer bleibt)

Inlet	Bedeutung
1	Pitch (Note) = Welcher Button
2	Velocity = Farbe/Helligkeit

Beispiel:

max

```
`[32 63]
|
[pack i i]
|
[noteout]
`
```

→ Button mit **Note 32** leuchtet **Amber voll** (63) auf **MIDI-Kanal 1**.

Kleiner Test:

max

```
`[button]
|
[prepend 63]    ← setzt Velocity = 63 (Amber voll)
|
[pack 32 i]     ← Button-ID = 32
|
[noteout]
```

 Du kannst das `63` durch andere Werte ersetzen, z. B.:

- 15 = Rot voll
- 60 = Grün voll
- 12 = Aus

Wenn du irgendwann **mehrere Templates oder Geräte** ansteuern willst, kannst du das dritte Inlet verwenden, aber für den Alltag reicht:

Pitch + Velocity = fertig. 

Willst du dir vielleicht ein kleines Interface basteln mit Farbknöpfen? Ich helf dir gern.



warum kommt von den potis kein wert bei notein herein ?

Sehr gute Frage – und das Verhalten ist **absolut korrekt**:

 **Die Potis (Drehregler) auf dem Launch Control XL senden keine Note-Werte, sondern Control Change (CC)-Werte.**

Warum?

MIDI unterscheidet:

- **[notein]** → empfängt **Note-On/Off** (also von Buttons, Pads etc.)
- **[ctlin]** → empfängt **Control Change (CC)** → das ist das, was Potis & Fader senden

Lösung in Max/MSP

Wenn du sehen willst, was von den Potis kommt:

max

```
`[ctlin]
|
[print potis]
```

 Wenn du an einem Poti drehst, bekommst du Ausgaben wie:

```
makefile
```

```
`potis: 21 64 1`
```

Das bedeutet:

- **21** = CC-Nummer (welcher Poti)
- **64** = Wert (z. B. 0–127, je nach Stellung)
- **1** = MIDI-Kanal (in Max: 1 = Kanal 2, da Max von 1 beginnt)

Vergleich:

Element	Empfänger in Max	MIDI-Typ
Buttons (Note-Modus)	<code>`notein`</code>	Note On/Off
Potis, Fader	<code>`ctlin`</code>	Control Change (CC)

Wie weiß ich, welcher Poti welche CC-Nummer sendet?

1. Öffne den **Launch Control XL Editor**
2. Lade dein aktuelles Template
3. Schaue dir die **CC-Nummern der Potis** an
4. Oder: einfach mit ``[ctlin]`` lauschen und selbst herausfinden

Wenn du willst, kann ich dir ein kleines Patch bauen, das dir live zeigt, **welcher Regler welchen CC-Wert sendet**, und vielleicht sogar eine visuelle Rückmeldung gibt. Interesse?



ok ich verwende also notein und ctlin um alle bedienelemente des controllers zu sehen

Ganz genau! 

Mit ``notein`` und ``ctlin`` zusammen siehst du **alle Bedienelemente** des Launch Control XL:

Übersicht – welche Objekte du brauchst:

Element	Objekt in Max	Beschreibung
Buttons	<code>`notein`</code>	Senden Note-On/Off (z. B. 32, 127)
Potis & Fader	<code>`ctlin`</code>	Senden CC + Wert (z. B. 21, 64)

Beispielpatch: Alles anzeigen

max

```

\----- [notein] -----
|
| [pitch]      [velocity]  [channel]
|               |           |
| [print note_pitch]
| [print note_velocity]
| [print note_channel]
|
|----- [ctlin] -----
|
| [cc#]        [value]     [channel]
|               |           |
| [print cc_number]
| [print cc_value]
| [print cc_channel]
|
\

```

Das zeigt dir:

- welche Taste du drückst (Note)
- welcher Regler bewegt wurde (CC)
- welchen Wert er hat
- und auf welchem MIDI-Kanal es gesendet wurde (meistens Kanal 1)

Bonus: Alternative kompakt

Wenn du nur mal schauen willst, was reinkommt:

max

```

\ [notein]
|
| [print note]
|
| [ctlin]
|
| [print cc]
|
\

```

Wenn du willst, bau ich dir ein Mini-GUI, das aufzeigt, was du gerade bewegst oder drückst. Sag einfach Bescheid.



was macht dann das midiparse objekt ?

Sehr gute Frage! ``midiparse`` ist ein **praktisches Tool**, das dir hilft, **alle eingehenden MIDI-Daten zu zerlegen** – aber **nur**, wenn du z. B. ein `[midiin]`-Objekt verwendest, das **roh alle MIDI-Daten** liefert.

Was macht ``midiparse``?

`midiparse` nimmt den Output von `[midiin]` (also **alle MIDI-Bytes**), und **filtert** sie automatisch nach:

- Note On/Off
- CC (Control Change)
- Program Change
- Pitch Bend
- Aftertouch
- usw.

Typisches Setup:

```
max

[midiin]
|
[midiparse]
|
+ noteout
+ ctlout
+ etc.
\
```

`midiparse` hat mehrere Ausgänge, z. B.:

1. **Note On/Off**
2. **CC**
3. **Program Change**
4. **Pitch Bend**
5. usw.

Beispiel:

```
max

[midiin]
|
[midiparse]
|
[route note cc]
\
```

Dann bekommst du z. B. am `note`-Ausgang:

```
nginx

pitch velocity channel
```

Oder am `cc`-Ausgang:

```
arduino
```

```
`cc# value channel`
```

● Wann brauchst du `midiparse`?

- Wenn du **alle MIDI-Daten eines Ports** lesen willst
- Wenn du **nicht weißt**, ob es Note, CC, Program Change o. ä. ist
- Wenn du z. B. ein einzelnes `midiin`-Objekt für **mehrere Typen von Daten** nutzen willst

● Wann brauchst du es nicht?

- Wenn du nur spezifisch Note-Daten willst: → `notein`
- Wenn du nur CC-Daten willst: → `ctlin`

✓ Fazit

Situation	Empfehlung
Du willst Buttons & Regler getrennt	`notein` + `ctlin`
Du willst alles auf einmal analysieren	`midiin` → `midiparse`

Wenn du willst, kann ich dir ein Beispiel bauen, das `midiparse` zeigt und Note + CC automatisch anzeigt. Wäre das interessant?



nein, mit diesen 3 objekten kann ich wirklich alles aus dem controller extrahieren

Ganz genau! ✓

Mit diesen drei Objekten hast du **volle Kontrolle über alles**, was der Launch Control XL sendet:

🔲 Die heiligen Drei:

Objekt	Funktion
`notein`	Buttons (Note On/Off)
`ctlin`	Potis & Fader (Control Change)
`midiin`	Rohdaten (nur nötig, wenn du <i>wirklich alles</i> sehen willst, z. B. Sysex oder ungewöhnliche Messages)

Aber wenn du nur mit den Standard-Bedienelementen arbeitest: ➡ `notein` + `ctlin` **reichen absolut aus**.



Beispiel-Setup für vollständigen Input vom Controller

```

max

\ ----- Buttons -----
[notein]
|
[print note]

----- Regler & Fader -----
[ctlin]
|
[print cc]
\

```

Damit kannst du:

- alle Buttons erkennen (auch gedrückt/losgelassen über Velocity 127/0)
- alle Regler + Fader verfolgen (mit CC-Nummer und Wert)
- MIDI-Kanal sehen (3. Outlet)

Wenn du später noch:

- LEDs steuern willst → ``noteout`` / ``ctlout``
- Templates wechseln willst → Sysex (aber brauchst du nicht unbedingt)

Dann hast du schon die **perfekte Basis** für ein eigenes Max-Interface.

Willst du jetzt einen nächsten Schritt machen, z. B.:

- eine **Mapping-Matrix** für Buttons?
- visuelle Anzeige der Werte?
- Farbsteuerung von LEDs?

Sag einfach Bescheid.